```
[1, 2, 3, 4] <=> [1, 2, 3] # => 1
[1, 2, 3] <=> [1, 2, 3]    # => 0
```

Some mathematical operators are defined for arrays. Adding two arrays returns a new array concatenating the two operands:

```
[1, 2, 3] + [4, 5, 6] # => [1, 2, 3, 4, 5, 6]
```

Subtracting two arrays returns a new array with every element in the first array that is not in the second array. This is aliased as difference:

```
[1, 2, 3, 4, 5] - [2, 4] # => [1, 3, 5]
```

You can multiply an array with an integer or string. The integer version returns that many copies of the array concatenated together. The string version returns each element of the array separated by the string—it's the same as join. Multiplying two arrays together to get the cross-product of all combinations of their elements is done with the product method:

```
[2, 4, 6] * 3                 # => [2, 4, 6, 2, 4, 6, 2, 4, 6]
[2, 4, 6] * ", "              # => "2, 4, 6"
[2, 4, 6].product(["a", "b"]) # => [[2, "a"], [2, "b"], [4, "a"], [4, "b"], [6,
                              # .. "a"], [6, "b"]]
```

Arrays also can do set operations. The intersection method takes an arbitrary number of other arrays as arguments and returns a new array of all elements that are in the original array and all the arguments. The resulting array will contain no duplicates, and will preserve the order of the elements. If there is only one other array being compared, you can use the & operator as a shortcut:

```
[1, 2, 3, 4, 5].intersection([1, 2, 3, 6], [-2, 2, 4]) # => [2]
[1, 2, 3, 4, 5] & [1, 2, 3, 6]                         # => [1, 2, 3]
```

Similarly, the union method takes an arbitrary number of array arguments and returns all elements that are in any of the arrays. Duplicates are removed, and the order of the elements remains the same. If there is only one other array, you can use | as a shortcut:

```
[1, 2].union([3, 5, 1], [12, -3, 2]) # => [1, 2, 3, 5, 12, -3]
[1, 2] | [3, 5, 1]                   # => [1, 2, 3, 5]
```

Ruby does combinatorics, too! The combination method takes an integer argument and returns an enumerator that will return every combination of elements in the array that is the size of the argument. If passed a block, it will yield the block to each combination. The permutation method does the same thing for every permutation—the difference being that order is significant in each permutation, but not in each combination:

```
[1, 2, 3].combination(2).to_a # => [[1, 2], [1, 3], [2, 3]]
[1, 2, 3].permutation(2).to_a # => [[1, 2], [1, 3], [2, 1], [2, 3], [3, 1], [3,
                              # .. 2]]
```

Both methods have a repeated form—repeated-combination and repeated-permutation—that allow the elements of the array to be repeated in one of the resulting values.

Arrays and Binary Search

Ruby has a binary search feature built into arrays. It isn't quite math, but it's algorithmic, and one day it will likely save you a great deal of time. The great thing about Ruby's binary